

关于 SGLang

SGLang 是一个让“多个大模型调用组成的程序”跑得更快、更省钱的系统。

它不是训练新模型，而是优化怎么调用大模型、怎么复用计算结果、怎么并行执行。

1. 它解决什么问题？

现在很多 AI 应用不是只问大模型一次，而是：多轮聊天，让模型查资料、推理、调用工具生成 JSON、表格等固定格式，一个程序里调用模型很多次，这些叫 LLM 程序。

产生的问题是：

传统系统每次调用都重新算一遍，很多重复内容被反复计算，结果就是：慢、贵、浪费显存。

2. SGLang 是什么？

SGLang 分成两部分：

① 前端语言

一种嵌入 Python 的小语言，写 LLM 程序更方便。

它提供一些“积木”：

gen：让模型生成内容

select：从几个选项里选

fork / join：并行执行多个分支

image / video：处理图片、视频

仍然可以用 Python 的循环、判断、函数。

② 后端运行时

真正负责加速执行的部分，叫 SGLang Runtime (SRT)。

它做了三个关键优化。

3. 三个关键加速技巧

技巧一：RadixAttention —— 复用 KV Cache

KV Cache 可以理解成：模型生成时记住的“中间计算结果”。

如果两个请求开头一样，比如都有同一段系统提示，那这段结果本来可以复用。

传统系统经常用完就扔。

SGLang 用一棵 radix tree（前缀树也叫基数树）把这些缓存管理起来：

相同开头 → 直接复用

显存不够 → 按 LRU 淘汰最久没用的

调度请求时 → 优先执行共享前缀长的

好处：减少重复计算，提高吞吐量，降低延迟。

技巧二：压缩有限状态机 —— 结构化输出更快

如果你要求模型输出 JSON、正则格式，传统做法是：

一个 token 一个 token 地检查

每次只能生成一个 token

SGLang 把规则变成 压缩有限状态机。

如果某一段是固定的，比如 {"summary": "，它就能一次生成多个 token。

实验里，JSON 解码吞吐量提升约 1.6 倍。

技巧三：API 推测执行 —— 黑盒 API 也省
有些模型只能通过 API 调用，比如 GPT-4。
一个程序如果分两次问，就要付两次输入费用。

SGLang 让第一次调用“多生成一点”，后面如果正好用得上，就省掉一次 API 调用。
这样能减少调用次数和输入 token 成本，实验里成本约降低 3 倍。

4. 效果怎么样？

论文在多种模型和任务上测试：

模型：Llama、Mixtral、LLaVA、GPT-3.5 等

任务：MMLU、ReAct、JSON 解码、多轮聊天、RAG、多模态等

硬件：A10G、A100G

结果：

吞吐量最高提升 6.4 倍

延迟最高降低 3.7 倍

缓存命中率可达 50%~99%

已经在 Chatbot Arena 部署，实际缓存命中率不错

关于 vLLM（重点在于 PagedAttention）

1. 它解决什么问题？

大模型生成文字时，每生成一个新 token，都要用到前面所有 token 的中间结果。这些中间结果存下来，就是 KV Cache。

KV Cache 很大，而且：每个请求长度不一样，输出长度事先不知道，会不断变长

传统系统要求它在显存里连续存放

结果就是：

预留太多 → 内部碎片

长度不一 → 外部碎片

不能共享 → 重复占用

论文实测：传统系统里真正用来存有效 KV Cache 的内存只有 20%~38%，大部分都浪费。浪费导致能同时处理的请求少，吞吐量上不去。

2. 核心方法：PagedAttention

灵感来自操作系统

操作系统用“虚拟内存 + 分页”解决内存碎片：把内存切成固定大小的页，逻辑页可以映射到不连续的物理页，按需分配，不用提前占一大块

vLLM 把这套思路搬到 KV Cache：把 KV Cache 切成固定大小的 KV block，每个 block 存固定数量 token 的 key/value，逻辑 block 通过 block table 映射到物理 block，物理 block 不需要连续。这样：几乎不浪费内存可以按需增长，可以共享，可以回收

3. 怎么工作？

基本流程

请求进来，先算 prompt 的 KV Cache。按 token 数量分配逻辑 block。block table 记录：逻辑 block → 物理 block。生成新 token 时，如果当前 block 没满，就写进去；满了就申请新物理 block。请求结束，物理 block 释放，给别人用。因为所有 block 大小一样，所以没有外部碎片；因为按需分配，所以内部碎片最多只有一个 block 的浪费。

4. 内存共享：省更多

vLLM 支持多种解码场景：

并行采样

同一个 prompt 生成多个答案。这些答案可以共享 prompt 部分的 KV Cache。只有后面不同的部分才单独存。写的时候用 copy-on-write：谁要改，谁复制一份。

Beam Search

保留 top-k 候选。不同候选之间可以共享大量前缀 block。共享模式动态变化，vLLM 用引

用计数管理。

共享前缀

比如系统提示、few-shot 例子。服务方可以提前把公共前缀的 KV Cache 存好。用户请求只要映射过去，只算自己的任务输入。

混合解码

不同请求可以用不同解码方式，vLLM 统一用逻辑块到物理块的映射层管理，方便一起批处理。

5. 调度与抢占

请求太多、显存不够时：采用 FCFS（先来先服务）要抢占时，先抢占最晚来的，一个序列的所有 block 要么全留，要么全换出，一个请求里的多个序列（如 beam）一起调度，被换出的 KV Cache 有两种恢复方式：

Swapping: 换到 CPU 内存

Recomputation: 之后重新算一遍

6. 分布式执行

大模型放不下一张 GPU 时，用张量并行拆到多张 GPU。vLLM 用一个集中调度器，维护统一的 block table。每张 GPU 只存自己负责的 attention head 那部分 KV Cache。调度器把 block table 广播给 GPU，GPU 按表读取。

7. 实验结果

模型：OPT-13B / 66B / 175B, LLaMA-13B

数据集：ShareGPT、Alpaca

对比：FasterTransformer、Orca

结果：

吞吐量提升 2~4 倍

长序列、大模型、复杂解码提升更明显

并行采样节省内存 6%~30%

Beam search 节省内存 37%~66%

共享前缀场景吞吐提升 1.67~3.58 倍

默认 block size 设为 16 比较合适